

Learning Mesh-Based Aerodynamic Simulation with Graph Networks

Project Proposal

Riccardo Castellano Xavier Plantier Sydney Daniel Fedez de la Mela

Machine Learning in Network Science - CentraleSupélec, March 2026

1 Motivation and Problem Definition

Simulating airflow around a wing or a drone is a central task in aerodynamic design, but high-fidelity CFD solvers are extremely expensive: a single simulation can take hours on a powerful machine, which makes iterative shape optimisation impractical. The question we want to explore is: *can a neural network learn to replicate these simulations much faster, by watching many of them and predicting the next flow state directly?*

This is the idea behind **MeshGraphNets** [1], a Graph Neural Network (GNN) framework that treats a simulation mesh as a graph and learns the physical dynamics from data, with no solver at inference time. The paper demonstrates results across four very different physical systems: cloth, structural mechanics, incompressible and compressible fluid dynamics, running 1–2 orders of magnitude faster than the original simulators (Figure 1).

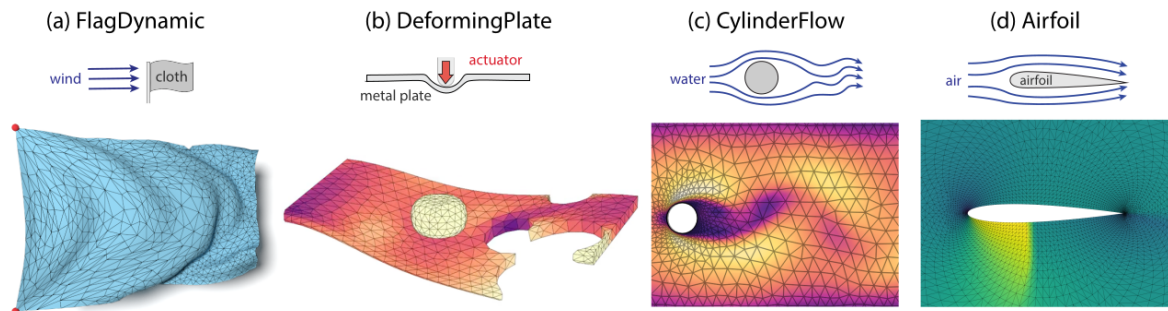


Figure 1: The four physical domains studied in MeshGraphNets [1]: flag dynamics, deforming plate, cylinder flow, and airfoil aerodynamics. Our project focuses on the airfoil case (rightmost).

Standard image-based models (CNNs) cannot handle CFD data naturally because meshes are *irregular*: cells are packed densely near the wing surface and sparse far away. GNNs are a natural fit: they treat each mesh node as a graph node and pass information along mesh edges, regardless of the mesh layout [2].

Why it matters. Faster aerodynamic surrogates are directly useful for drone design, aircraft shape optimisation, and real-time flight simulation. More broadly, the MeshGraphNets framework is general enough to apply to any mesh-based physical system, making it a particularly interesting model to study.

Related work. Early ML surrogates for CFD used CNNs on regular grids [5, 6], which struggle with adaptive meshes. Particle-based GNN simulators [4] handle irregular data but ignore mesh connectivity. Some works embed a PDE solver in the loop [3], improving accuracy at the cost of complexity. MeshGraphNets takes a cleaner, solver-free approach and outperforms these baselines.

2 Methodology

2.1 Implementing MeshGraphNets

Our main goal is to implement MeshGraphNets and apply it to the airfoil case. The model follows a three-step **Encode–Process–Decode** pipeline (Figure 2):

1. **Encode:** The mesh is converted into a graph. Nodes carry physical quantities (velocity, pressure, density); edges carry geometric information (relative positions). Everything is compressed into compact latent representations.
2. **Process:** The graph goes through 15 rounds of *message passing*, where each node collects information from its neighbours and updates its own state. Extra “world-space” edges are added between nodes that are physically close but far apart on the mesh, helping the model capture non-local effects like shocks or wakes.
3. **Decode:** Each node’s latent state is mapped to a physical update how much does velocity and density change in the next step? These updates are accumulated autoregressively to produce a full simulation trajectory.

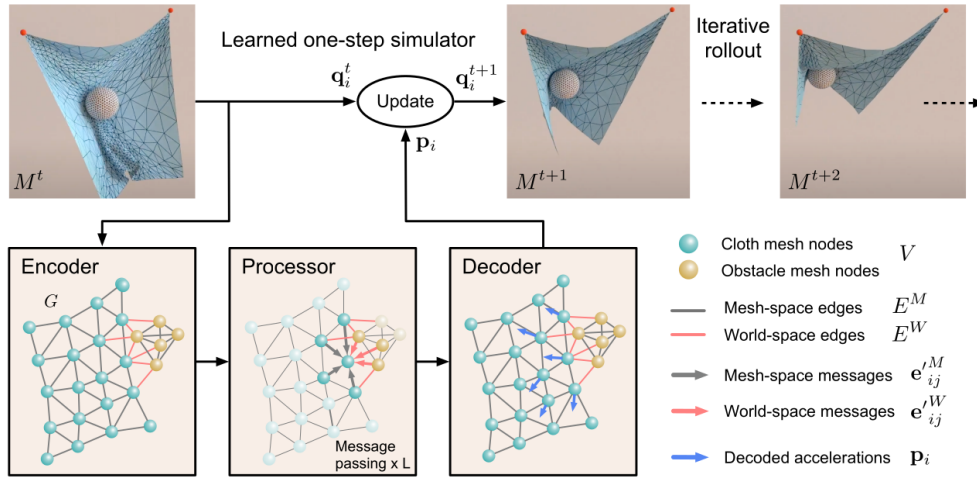


Figure 2: The Encode–Process–Decode architecture of MeshGraphNets. The mesh at time t is encoded as a graph, processed through repeated message-passing rounds, and decoded to produce the next mesh state M^{t+1} .

2.2 Simulation Data and Two-Phase Plan

A note on fluid dynamics simulations. Setting up a CFD simulation correctly is notoriously delicate: a wrong mesh, boundary condition, or solver setting can produce physically incorrect results, and the model would then learn from bad data. For this reason, we chose to ground our work in the **official 2D airfoil tutorial** included in OpenFOAM a well-validated, community-maintained benchmark. We ran this simulation ourselves (Figure 3) and obtained physically consistent results: the expected pressure gradient over the wing, correct stagnation at the leading edge, and a well-formed wake behind the trailing edge. This gives us confidence in the quality of any data we generate with this setup.

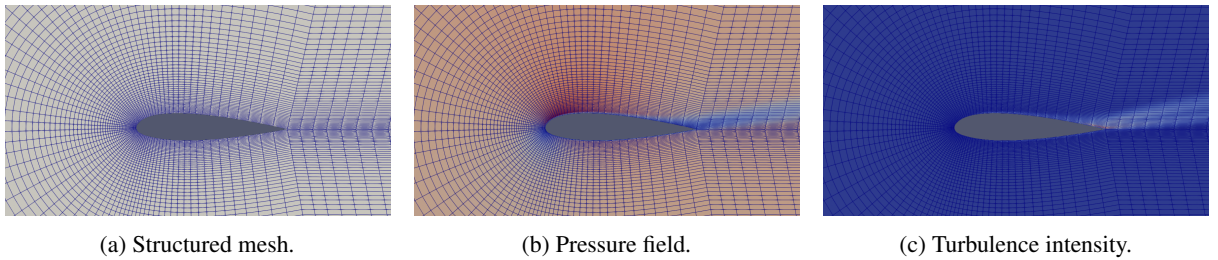


Figure 3: Our OpenFOAM 2D airfoil simulation. Starting from this validated tutorial case ensures that the training data is physically meaningful.

Phase 1 — DeepMind AIRFOIL dataset. We will first train and evaluate on the publicly available AIRFOIL dataset released with [1]: 2D compressible flow trajectories at varying angles of attack and inflow speeds, on adaptive meshes with 5200 nodes. This is our baseline and validation step.

Phase 2 — Custom OpenFOAM data (extension, time permitting). If Phase 1 goes well, we hope to generate our own trajectories using our OpenFOAM setup, varying the NACA profile or flight conditions, and test whether the model generalises to unseen geometries.

Training. We will follow the original protocol: one-step supervised training with an L_2 loss, Adam optimiser ($\text{lr} = 10^{-4}$), and Gaussian noise on inputs to improve rollout stability.

3 Evaluation

Metrics. We will use the same evaluation as the original paper: 1-step prediction error, 50-step rollout error, full-trajectory error (600 steps), and inference speedup against the OpenFOAM solver. We expect the GNN to be significantly faster and are curious to see how close we can get to the speedups reported in the paper with our own implementation.

Baselines. We will compare against a UNet on a regular grid interpolation of the mesh, and a standard GCN [7] with the same encode-decode wrapper following the ablation study of [1].

Generalisation. We will test on conditions outside the training range (steeper angles, higher Mach numbers) to assess whether the model has learned something physically meaningful or is memorising training examples.

Datasets. **Primary:** DeepMind AIRFOIL dataset (public). **Extension:** Custom OpenFOAM trajectories on new NACA profiles (time permitting).

References

- [1] T. Pfaff, M. Fortunato, A. Sanchez-Gonzalez, P.W. Battaglia. Learning Mesh-Based Simulation with Graph Networks. *ICLR*, 2021.
- [2] P.W. Battaglia et al. Relational Inductive Biases, Deep Learning, and Graph Networks. *arXiv:1806.01261*, 2018.
- [3] F. de Avila Belbute-Peres, T.D. Economou, J.Z. Kolter. Combining Differentiable PDE Solvers and Graph Neural Networks for Fluid Flow Prediction. *ICML*, 2020.
- [4] A. Sanchez-Gonzalez et al. Learning to Simulate Complex Physics with Graph Networks. *ICML*, 2020.
- [5] X. Guo, W. Li, F. Iorio. Convolutional Neural Networks for Steady Flow Approximation. *ACM SIGKDD*, 2016.
- [6] S. Bhatnagar et al. Prediction of Aerodynamic Flow Fields Using Convolutional Neural Networks. *Computational Mechanics*, 64(2):525–545, 2019.
- [7] T.N. Kipf, M. Welling. Semi-Supervised Classification with Graph Convolutional Networks. *ICLR*, 2017.